Creating a DAS Listener in PowerShell

Reference RA 473

Why PowerShell?

PowerShell is a powerful and flexible scripting language built on the .NET platform. It is widely used for automation, system administration, and cross-platform scripting.

Choosing PowerShell for implementing a DAS (Data Analytics Solution) listener provides several benefits:

- **Native HTTP server support** via `HttpListener`
- **Built-in JSON parsing and concurrent collections**
- **Asynchronous task execution** with jobs and queues
- **Cross-platform compatibility** on Windows, Linux, and macOS via PowerShell Core

This makes PowerShell an excellent choice for building lightweight network listeners or service simulators for testing environments.

General Technical Info

Component	Version
Language	PowerShell
Framework	5.1+ / Core
Environment	Windows, Linux, macOS

Overview of the Program Structure

This RA provides a simple but effective implementation of a DAS message listener using the following components:

- `DASMessage.ps1`
  A class definition used to wrap incoming messages into structured objects containing the URL, the message name, and the JSON body.

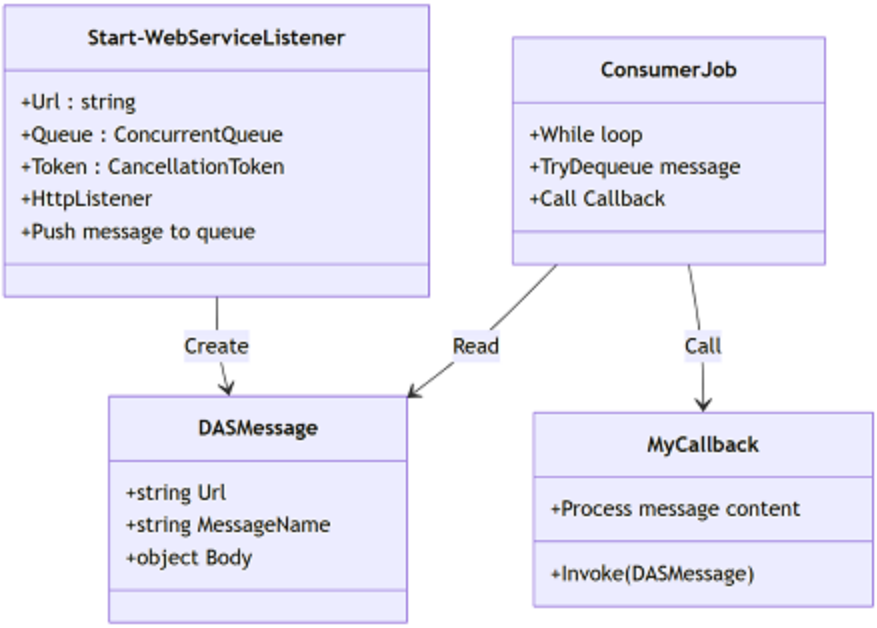
- `DASListener.ps1`
  A function that starts a local web listener, accepts HTTP messages, and enqueues them for processing using a concurrent queue.

- `CallbackExample.ps1`
  A sample function to be executed whenever a message is received. It demonstrates how to handle and inspect incoming data.

- `Run-DAS.ps1`

  A script to launch the DAS listener and message consumer interactively.
  It displays messages in real-time and stops when you press `q`.
  Logs are saved to `$env:TEMP\\das.log` if enabled. This modular architecture makes it easy to expand, reuse, and integrate into various AMP/DAS validation and testing flows.

Architectural Diagram



This documentation is part of the Teradyne Archimedes Reference Architecture series.

Advanced Details - PowerShell DAS Listener

Reference RA 473

> **Disclaimer:** This code is provided for instructional purposes only. It is not optimized for production use.

Execution Flow

1. Create a queue and cancellation token.
2. Start a consumer job to process messages.
3. Launch the listener on the specified URL.
4. Push each received HTTP message into the queue.
5. Process messages via the callback.
6. Gracefully stop with `$cts.Cancel()`.

Initialization Requirement and Error Handling

The listener expects the first message to be of type `INITIALIZATION`. If it is not, the listener should return an HTTP 500 error. This ensures the proper startup sequence and helps detect misconfigured test flows early.

Detailed Script Explanation

DASMessage.ps1

This file defines the `DASMessage` class, which encapsulates each HTTP message the listener receives. It includes:

- `Url`: the full request path
- `MessageName`: the final segment of the URL (used as the message identifier)
- `Body`: the parsed JSON body from the request

This object is passed to the callback function for processing.

DASListener.ps1

This is the core of the system and contains the listener logic:

1. **HttpListener Setup**: Initializes and starts an HTTP listener using the given URL prefix.
2. **Message Reading**: Waits for a client connection, reads the HTTP request body, and parses it as JSON.
3. **Message Wrapping**: Wraps the request in a `DASMessage` object and enqueues it into a concurrent queue.
4. **Queue Processing**: Processing is delegated to a separate job (see `how-to-use.md`) that dequeues and calls a callback function for each message.
5. **Graceful Shutdown**: Supports stopping the listener with a cancellation token.

`CallbackExample.ps1`

This script defines a sample callback function:

```
function MyCallback {
    param ($msg)
    Write-Host "==== Message Received ===="
    Write-Host "URL         : $($msg.Url)"
    Write-Host "MessageName : $($msg.MessageName)"
    Write-Host "Body        : $($msg.Body | ConvertTo-Json -Depth 5)"
    Write-Host "=========================="
}
```

You can replace `MyCallback` with any function that accepts a `DASMessage` object. This allows flexible message handling (e.g., saving to file, reacting to `INITIALIZATION`, routing to submodules).

`Run-DAS.ps1`

A script named `Run-DAS.ps1` is provided to simplify usage:

- It starts the listener and a consumer.
- It logs and displays messages received in real-time.
- It stops gracefully when the user presses `q` and Enter.

This is useful for interactive local testing without manually launching each component.

Design Note

- All message processing is **asynchronous** and **decoupled** from the HTTP listener.
- Errors in JSON parsing are caught and do not crash the listener.
- Using `ConcurrentQueue` ensures thread safety.
- The system is **modular** and can be expanded with additional message handlers or logic in the callback.

This documentation is part of the Teradyne Archimedes Reference Architecture series.

How to Use the PowerShell DAS Listener

Reference RA 473

This guide explains how to use the PowerShell scripts provided in RA 473.

Prepare the Environment

Make sure you have:

- PowerShell 5.1 or PowerShell Core installed
- Edit the AMP configuration file to declare the DAS

Configuration File

You need to declare your DAS in the following file:

```
C:\ProgramData\Teradyne\Production\AMP\Conf\DAS_Conf.xml
```

Example Configuration

```
<DAS>
    <URL>http://127.0.0.1:1000/TestDAS/</URL>
    <MaxMsg>50</MaxMsg>
    <Name>TestDAS</Name>
    <Enable>true</Enable>
    <Specification>S_08</Specification>
</DAS>
```

Ensure that the URL defined in the DAS matches exactly the one you provide to your listener.

Load the Required Files

Open a PowerShell console and run:

```
. ./source/DASMessage.ps1
. ./source/DASListener.ps1
```

```
. ./source/CallbackExample.ps1
```

Start the Listener

```
$queue = [System.Collections.Concurrent.ConcurrentQueue[object]]::new()
$cts = [System.Threading.CancellationTokenSource]::new()

$consumerJob = Start-Job -ScriptBlock {
    param ($queue, $callbackScript, $token)

    while (-not $token.IsCancellationRequested) {
        if (-not $queue.IsEmpty) {
            $ok, [ref]$msg = $null
            $ok = $queue.TryDequeue([ref]$msg)
            if ($ok) {
                & $callbackScript.Invoke($msg.Value)
            }
        } else {
            Start-Sleep -Milliseconds 100
        }
    }
    Write-Host "Consumer stopped."
} -ArgumentList $queue, ${function:MyCallback}, $cts.Token

Start-WebServiceListener -Url "http://localhost:8080/das/" -Queue $queue -Token $cts.Token
```

Stop the Listener

```
$cts.Cancel()
Stop-Job $consumerJob
Remove-Job $consumerJob
```

Using the Automation Script

Instead of running the steps manually, you can simply launch:

```
./source/Run-DAS.ps1
```

The script will:

- Start the DAS listener and background consumer
- Print all messages received
- Stop when you type `q` and press Enter

Logs are saved to `$env:TEMP\das.log` if write access is available.

This script is ideal for developers wanting to experiment with DAS-style communication and simulate AMP system behavior using native PowerShell.

This documentation is part of the Teradyne Archimedes Reference Architecture series.

Description	Download Link
Entire Solution	The solution
GIT Hub Link	Git Hub

More details about those source code here

This documentation is part of the Teradyne Archimedes Reference Architecture series.